Ticket To Ride
Design Document

Authors:
Scott Nguyen
Alan Sebastian
Neil Pereira

Team: SNA Solutions

Document Revision History

Date	Version	Description	Author
18/03/26	0.0	Created team contact table and filled in my details	Scott Nguyen
18/03/26	0.1	Added tech stack description and justification	Scott Nguyen
18/03/26	0.2	Added team schedule details	Scott Nguyen
20/03/26	0.3	Initial draft of concept descriptions for the Domain Model	Scott Nguyen
21/03/26	0.4	Added my personal information such as strengths and weaknesses into the doc	Alan Sebastian
21/03/26	0.5	Added Dynamic Model for GameSetup for the start of the game	Alan Sebastian
21/03/26	0.6	Added alternate Domain Models	Scott Nguyen
25/03/26	0.7	Added Dynamic Model for End Game Trigger and Final score	Neil Pereira
25/03/26	0.8	Added reasons why alternate Domain Models rejected and assumptions and constraints	Scott Nguyen
27/03/26	0.9	Added Domain Model and justification	Scott Nguyen
27/03/2026	1.0	Modified the Dynamic Model for GameSetup to remove unnecessary Function Names	Alan Sebastian
27/03/2026	1.1	Filled my section of team details	Neil Pereira
27/03/2026	1.2	Added Document Introduction	Neil Pereira

Contents

1 Introduction

This document presents the design of a digital implementation of *Ticket to Ride*, outlining the system structure, gameplay mechanics, and key design decisions derived from the official board game rules. The purpose of this document is to provide a clear and structured overview of how the system is modelled and how it will be implemented.

The core problem addressed in this project is the translation of a complex, rule-based physical board game into a consistent and well-defined digital system. *Ticket to Ride* involves multiple interacting components, including players, routes, cards, and scoring systems, all governed by strict rules and constraints. Modelling these elements in software requires careful consideration to ensure that game logic is enforced correctly, player actions are limited appropriately, and the overall gameplay flow remains faithful to the original experience.

The intended audience includes developers, tutors, and other stakeholders involved in the project. It serves as both a technical reference for implementation and a justification of the design choices made throughout the development process.

The document is organised into several sections. It begins with a team introduction, followed by domain modelling which defines the core entities and relationships of the system. This is then extended by the dynamic model, which illustrates system behaviour and interactions over time. Supplementary sections provide additional context, including assumptions, constraints, and alternative design considerations.This document is supported by the official *Ticket to Ride* rulebook, which defines the rules and constraints that guide the system design.

2 Team Introduction

2.1 Team Name and Team Photo (important parts of team building)

The team name is SNA Solutions.

Left to right: Neil Pereira, Scott Nguyen, Alan Sebastian

2.2 Team Membership

Name	Contact Details	Strengths	Fun Fact
Scott Nguyen *(Sprint 1 Lead)*	Email - sngu0065@student.monash.edu Discord - dodgyscotty	Technical - Proficient at Java, Python and Javascript - Familiar with OOP design principles Professional - Effective communicator - Strong organisation skills - Always learning	I can drive a forklift
Alan Sebastian	Email - aseb0007@student.monash.edu Discord - alansebastian_	Technical - Proficient at Java and Python - OOP design principles Professional - Strong organisation and teamwork skills - Communicates effectively	I can solve a rubix's cube
Neil Pereira	Email - nper0041@student.monash.edu Discord - pereira_neil	Technical - Proficient at Java, Python and C++ - OOP design principles Professional - Strong problem-solving skills - Receptive to feedback and continuous improvement	Im a black belt in karate

2.3 Team Schedule

The team meets once a week in class on Wednesdays at 10-12 am. If for any reason the team is unable to meet during that time, then the discussion is held online via the discord group chat.

The team has agreed upon a work schedule built on trust, whereby each member can do the work at their own pace as long as it gets done by a date and time agreed by each member. Dates and times when work is expected to be finished are usually decided during each week's meeting.

The team strives to give a balanced workload to each member. At each meeting, deliverables are discussed and the work it entails is estimated. The work is then distributed based on these estimates. If the work assigned to a member is too much, then other members will help them to complete it. The team regularly communicates via online chats to give updates on work and ask for help and feedback.

2.4 Technology Stack and Justification

The team decided to implement the Ticket To Ride game using the Java tech stack. All of our team members are proficient with Java due to experience using it for another unit, FIT2099. While each member has also used Python in various other units and side projects, it was decided that since the team learnt Java in an object oriented style in FIT2099, it was advantageous to use it in this project which demands an object-oriented style of programming. Additionally, not all of the team has used Python in an object-oriented manner before as well, which solidified the decision to go with Java.

As for the implementation of the user interface, the team has had no prior experience using Java Swing. The team will utilise tutorials, online documentation and artificial intelligence to upskill in Java Swing in order to create a functional, usable and presentable user interface. One of the team members has experience with Python's TKinter, however since the team has strong object-oriented programming skills with Java, we anticipate that learning Java Swing should not be much of a hurdle. If anything, if Python was chosen, learning TKinter would be a bigger hurdle, meaning it would take longer to make the project and it would be lacking in quality.

2.5 Link to the Gitlab repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/CL_Wednesday10AM_04

**Some commits were made under Scott Nguyen (33879095)'s personal git account 'scottnguy3n101'*

**Some commits were made under Neil Pereira (35539666)'s personal git account 'Neilsbp'*

3 Domain Modelling

3.1 Domain Model

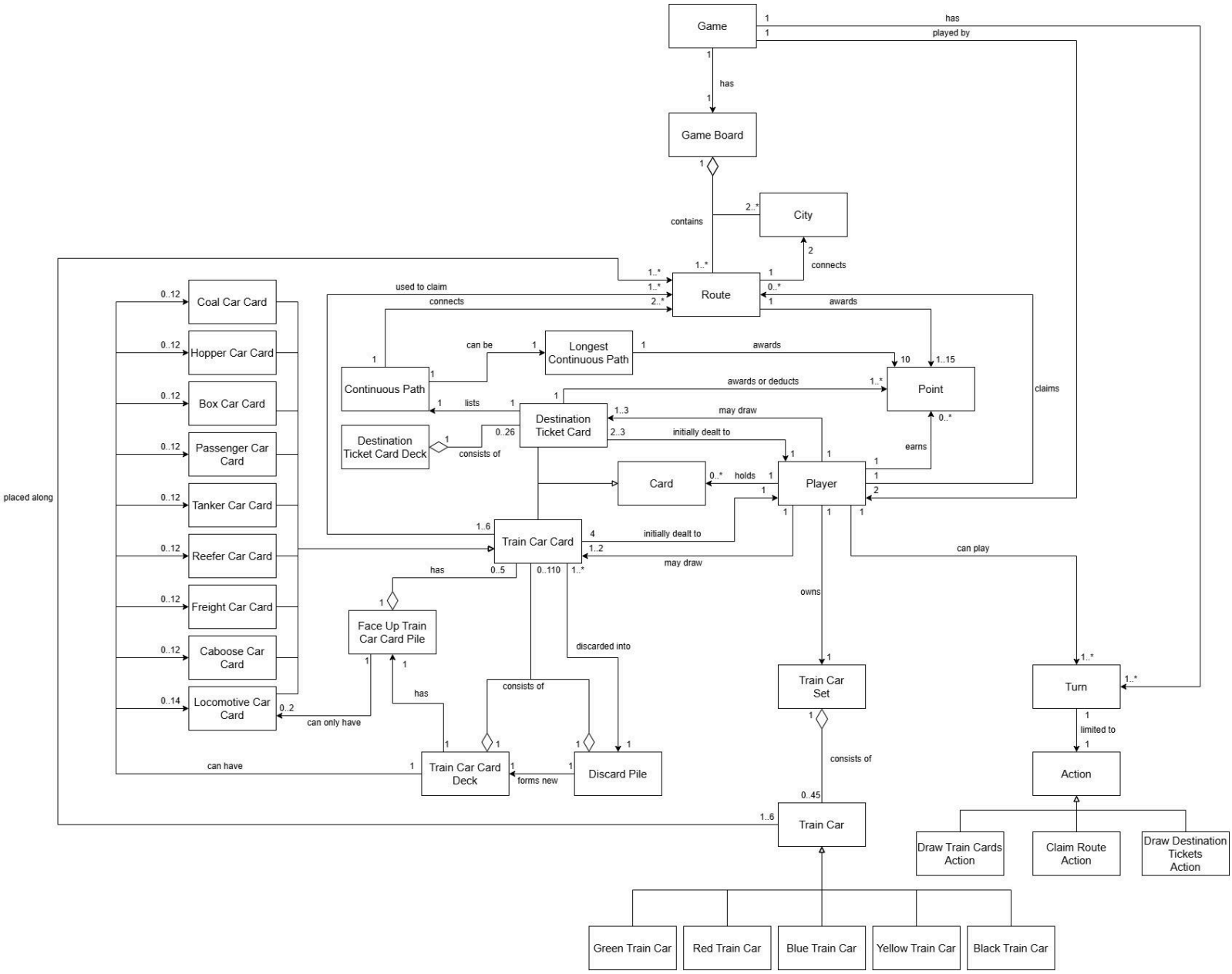

Figure 3.1.1 - Final Domain Model of the Ticket To Ride Game. The jpeg of the diagram can be found in the git repository.

The final Domain Model can be seen in Figure 3.1.1. It encapsulates all relationships related to Ticket To Ride gameplay and rules like turn behaviour, where cards can end up etc. The entity Card was used as an abstraction layer for the Train Car Cards and Destination Ticket Cards as they can both be in a Player's hand. However, the same was not done for the Ticket Card Deck and Destination Ticket Card Decks by adding an entity called Deck, as both decks are composed of different numbers of cards and the decks do not mix the types of cards together, so adding a Deck entity would introduce unnecessary complexity.

In addition, the different types of Train Car Cards like the Caboose, Freight Car etc were included in the diagram as it was important to demonstrate how many of these cards may exist in the game. The association between Train Car Card Deck and each type of Train Car Card indicates the max number of those cards in the cardinality. This helps with implementation by identifying constraints. The inclusion of Locomotive Car Card was especially helpful as it is different to the other Train Car Cards as there are 14 available (not 12 like the others) and also that only two may exist in the face up pile. These differences are captured in the Domain Model above.

In this diagram, instead of an association from Player directly to Train Car, the entity Train Car Set is placed in between. This more accurately represents the gameplay where Players grab a set and not individual Train Cars. It also indicates that Players collect a specific set of Train Cars of one colour, rather than picking up Train Cars of different colours.

The Turn, Action, and three types of Action entities were included to clarify the things a Player may do in a turn. Without these entities, associations and cardinalities, the diagram could be interpreted as Players may draw Destination Ticket cards, Train Car cards and claim a Route all in one turn. So by having the entity Turn have a one-to-one association with Action, the rule that Players can only do one action per turn is enforced. The diagram has the associations related to these different Actions attached to the relevant entities rather than the Action itself. For example, there is no association from 'Draw Train Car Cards Action' to 'Train Car Card' that says the action draws 2 Train Car Cards. Instead this is captured through an association between Player and Train Car Card. The reason is because the Action itself does not draw the cards, but the Player is the one that draws them.

The different coloured Train Cars were included in this diagram, like the Green Train Car, to provide developers the constraint that there can be only Green, Red, Blue, Yellow or Black train cars. This prevents implementations from including other colours.

This diagram also omits the scoring track and scoring marker in the physical version of Ticket To Ride as they are not relevant to the problem domain of a digital version of Ticket To Ride which this model is aiming to help develop. Instead, the Point entity keeps track of points in the game.

Also, one of the rules is if a Locomotive card is drawn from the face up pile of the Train Car Card deck, then a second card cannot be drawn. This was implicitly shown in the association Player may draw 1..2 Train Car cards.

Alternate versions of the Domain Model are discussed in *Section 5 - Additional Documentation*.

3.2 Concept Descriptions

3.2.1 Game Map Entities

Concept: Board Map

Description: This is where the whole game is played

Relationships:

- Contains at least one Route connecting two Cities
- Only one may exist in a single Game

Concept: Route

Description: A line that connects two cities on the game board

Relationships:

- Connects 2 cities on the map
- Can be claimed by one or more Train Car Cards
- Once claimed, Train Cars matching colour and number of the Route are placed along it
- A Player can claim multiple Routes
- Routes can award 1-15 points when claimed
- Routes can connect to each other to form Continuous Paths

Concept: City

Description: A point on the map, connected to another by a Route

Relationships:

- Two or more cities can exist on the game board
- A route connects two cities

Concept: Continuous Path

Description: A path comprising two or more routes connected end to end.

Relationships:

- Connects 2 or more routes on the map
- Only one in any game can be considered the Longest Continuous Path
- A Destination Ticket card lists one Continuous Path

Concept: Longest Continuous Path

Description: The longest continuous path in a game

Relationships:

- Only one Continuous Path may be considered the Longest in a single game
- The Longest Continuous Path awards 10 points

3.2.2 Cards

Concept: Card

Description: A card in the game. Can either be a Train Car Card or Destination Ticket Card.

Relationships:

- General form of cards in the game used to encapsulate both Train Car and Destination Ticker cards
- Players may hold any amount of these

Concept: Train Car Card

Description: Cards that represent a Train Car of a specific colour. Comes in sets.

Relationships:

- 110 total, 12 each of Box, Passenger, Tanker, Reefer, Freight, Hopper, Coal, and Caboose. Plus 14 Locomotives.
- Players can hold zero or more of them
- Players initially draw 4 Train Car Cards at the start of the game
- In a turn, Players can draw 2 Train Car Cards. They can only draw one if the first is a Locomotive from the face up pile.
- Zero or more may be in the Train Car Card Deck, up to 110
- Zero or more may be in the discard pile, up to 110
- A combo of 1 or more can match one or more Routes

Concept: Coal Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Hopper Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Box Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Passenger Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Tanker Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Reefer Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Freight Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Caboose Car Card

Description: A type of Train Car Card

Relationships:

- Max of 12 in a deck

Concept: Locomotive Car Card

Description: A special type of Train Car Card

Relationships:

- Max of 14 in a deck
- Only two allowed in the face up pile of a deck
- If drawn from face up pile, unable to pick a second card

Concept: Train Car Card Deck

Description: A deck of Train Car cards

Relationships:

- Can consist of 0 to 110 Train Car Cards
- When empty, the Discard Pile is shuffled into a new Train Car Card Deck
- A Train Car Card Deck has a pile of face up Train Car Cards, there are always 5 of these until the deck size reaches below 5.
- A Deck can have any amount of each type of Train Car Card up to the max available, for example they may only be up to 12 Caboose Cars as only 12 exist in a game.

Concept: Discard Pile

Description: The pile of discarded Train Car Cards after they were played

Relationships:

- Can consist of up to 110 Train Car Cards
- One or more Train Car Cards are discarded into this pile after being used to claim a Route
- The Discard Pile is reshuffled to become the Train Car Card Deck if the Deck has 0 cards left

Concept: Face Up Train Car Card Pile

Description: The portion of the Train Car Card Deck that contains the face up Train Car cards

Relationships:

- Must always have 5 face up Train Car Cards, but if Train Car Card Deck goes below 5, face up cards equal to size of Deck until 0.
- Only one face up pile exists in the Deck

- A Face Up Train Car Card Pile may only have up to two Locomotive Cards. If above two, existing Face Up Card Pile must be reshuffled into the Train Car Deck and a new Face Up Train Car Card Pile is formed

Concept: Destination Ticket Card

Description: Cards that represent certain Continuous Paths on the game board

Relationships:

- 30 total, with up to 26 being in the Destination Ticket Card Deck
- Zero or more of these can be held by a player
- One Destination Ticket Card contains one Continuous Path
- They can award or deduct points depending on if the player completes the Continuous Path it contains or not.
- Players initially draw 3 Destination Ticket Cards, but may return one of them
- Players can draw 3 Destination Ticket Cards in a turn, and may return one or two of them.

Concept: Destination Ticket Card Deck

Description: The deck of Destination Ticket cards

Relationships:

- Can be comprised of zero up to a max of 26 Destination Ticket cards with two players as they must hold two each at the start of the game

3.2.3 Game and Player Entities

Concept: Game

Description: The entity representing the main game loop

Relationships:

- Consists of a single game board
- Two players per game
- A Game can have multiple Turns

Concept: Player

Description: One of the people playing the Ticket To Ride game. They are responsible for the direction of the game and are the ones behind each action in the game.

Relationships:

- Takes a set of Train Cars
- Scores 0 or more points
- Can hold any amount of Train Car and Destination Ticket cards
- In a turn, can either draw 1 or 2 Train Car Cards (depending on if the first is a Locomotive from the face up pile), draw up to 3 Destination Ticket Cards (needs to keep 1, but can return the other two) or claim a Route.
- Only 2 Players exist in a Game session
- At the start of the Game, draws 4 Train Car Cards and 3 Destination Ticket Cards. They may return one of the Destination Ticker Cards drawn.
- Plays one or more Turns in a game

Concept: Turn

Description: When a Player makes a move in the Game. Turns are taken sequentially, player to player until the end of the game.

Relationships:

- A Game has two or more Turns
- Players play one or more Turns
- During a Turn, a Player is limited to one Action

Concept: Points

Description: Earned by Players during specific actions like claiming a route. Used to determine the winner of the game

Relationships:

- Players can earn multiple points
- Destination Ticket Cards award or deduct points depending on if the Player completed the Continuous Path on it or not
- The Player with the Longest Continuous Path is awarded 10 points
- When Routes are claimed, awards 1-15 points depending on length

Concept: Action

Description: The thing that a Player does during a Turn.

Relationships:

- Only one Action per Turn is allowed
- Three total actions possible, either drawing Train Car or Destination Ticket Cards or claiming a Route

Concept: Draw Destination Ticket Card Action

Description: When a Player draws Destination Ticket Cards in a turn

Relationships:

- Is a form of Action

Concept: Draw Train Car Card Action

Description: When a Player draws Train Car Cards in a turn

Relationships:

- Is a form of Action

Concept: Claim Route Action

Description: When a Player claims a Route in a turn

Relationships:

- Is a form of Action

Concept: Train Car

Description:. A coloured train car used to signify claimed routes, as players place them along routes when they claim them using Train Car Cards

Relationships:

- 240 total, with 45 in blue, red, green, yellow, and black plus replacement cars in each colour
- 45 Train Cars of one colour are in a single Train Car Set
- One or more placed along a Route once it is claimed (matching Route length)

Concept: Train Car set

Description:. A set of 45 Train Cars of a specific car

Relationships:

- Total of 5 sets, of blue, red, green, yellow and black
- One set per Player

Concept: Green Train Car

Description: A Green Train Car

Relationships:

- One of the possible colours a Train Car comes in

Concept: Red Train Car

Description: A Green Train Car

Relationships:

- One of the possible colours a Train Car comes in

Concept: Blue Train Car

Description: A Green Train Car

Relationships:

- One of the possible colours a Train Car comes in

Concept: Yellow Train Car

Description: A Green Train Car

Relationships:

- One of the possible colours a Train Car comes in

Concept: Black Train Car

Description: A Green Train Car

Relationships:

- One of the possible colours a Train Car comes in

3.3 Assumptions and Domain Constraints

- A game board must always have at least one route connecting two cities
- Routes in the game board must always have a length of at least one and max of 6 (as seen in the routes-points diagram in the instruction booklet)
- Players can finish the game with no Routes claimed, and so no points gained
- Continuous Paths must connect at least two Routes
- The maximum number of Train Car Cards that can be in the Train Car Cark Deck is 110. This is because there are a total of 110 Train Car Cards, and any number below 110 in the deck, especially when the Discard pile is reshuffled into a new Deck.
- Similarly, the maximum number of Train Car Cards in the Discard pile is 110, as it is possible all Train Car Cards end up being discarded during the game.
- It is also possible for the Train Car Card Deck and the Discard Pile to have 0 Train Car Cards, as it is clearly stated in the Ticket To Ride rulebook that this is possible if players hoard cards, but very unlikely.
- In the Train Car Card deck, for each type of card, it is possible for the number of each card in the deck to be the maximum available. For instance, there can be 14 Locomotive cards in the deck which is the total possible.
- The maximum number of Destination Ticket Cards in a Destination Ticket Card Deck is 26, as Players must keep 2 each at the start of the game. Unlike Train Car Cards, Destination Ticket Cards are kept in hand until the end of the game.
- Players must always take a replacement face up card when drawing from the face up pile
- The number of face up cards in the Train Car Card deck must always be 5. Since Players replace every face up card they draw. The only exception to this is when the deck size goes below 5, the deck would be full of face up cards with a number from 0-5.
- The number of Locomotive cards possible in the face up pile is 2, as a new face up pile must be made if there are 3.
- All destination ticket cards give continuous paths a point value of one or more.

4 Dynamic Model

Scenario Name: Game Setup

Scenario Description:

This scenario shows how the system initializes a new Ticket to Ride game, distributes starting resources, allows each player to choose which destination tickets to keep, and transitions the game into the ready state for the first turn.

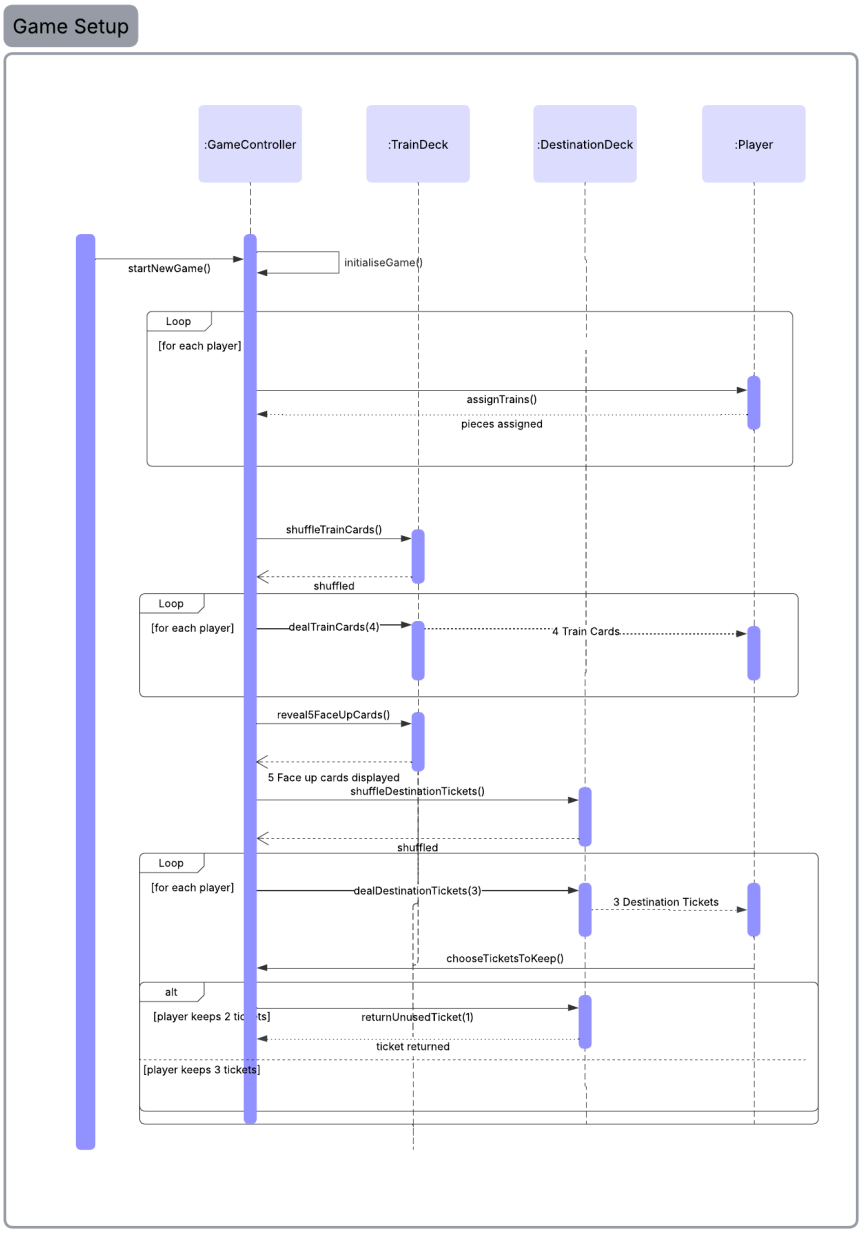

Figure 4.1 - Dynamic model for Game Setup

Scenario Name: End Game Trigger and Final Scoring

Scenario Description:

At the end of the active player's turn, the system checks whether that player has 2 or fewer trains remaining. If yes, the game enters the final round, all players receive one last turn, then the system reveals destination tickets, applies ticket points, awards longest continuous path bonus, and determines the winner.

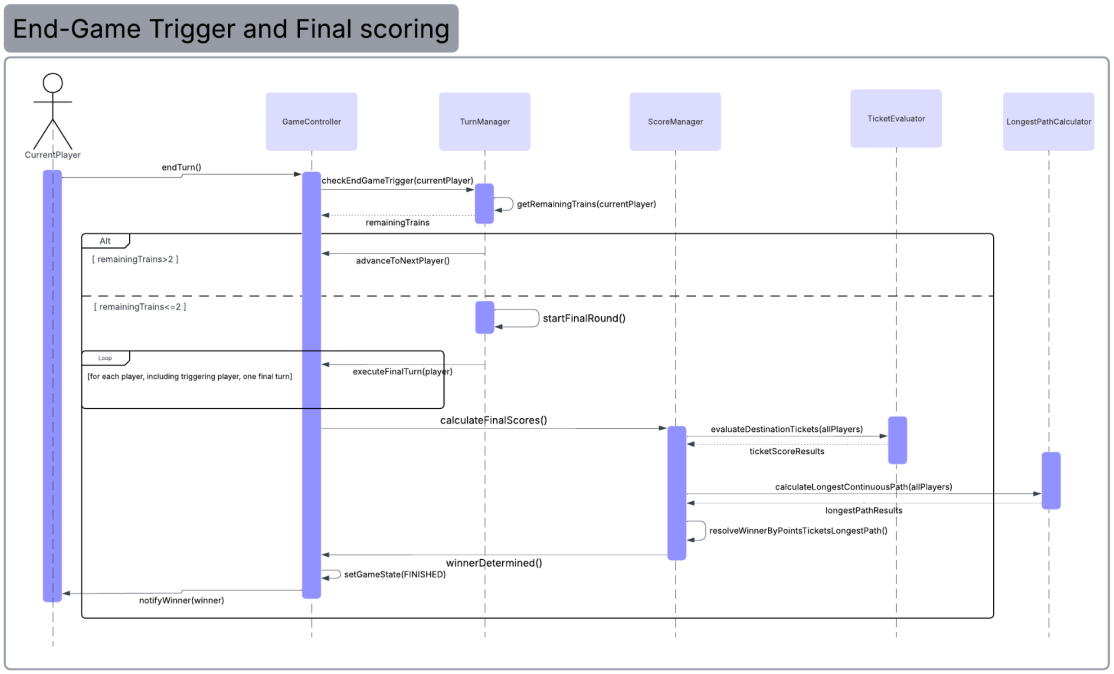

Figure 4.2 - Dynamic model for End-Game Trigger and Final Scoring (lucid.app link)

5 Supplementary Documentation

5.1 Alternate Domain Models

The domain models listed below were past iterations of the final domain model that the team eventually reached.

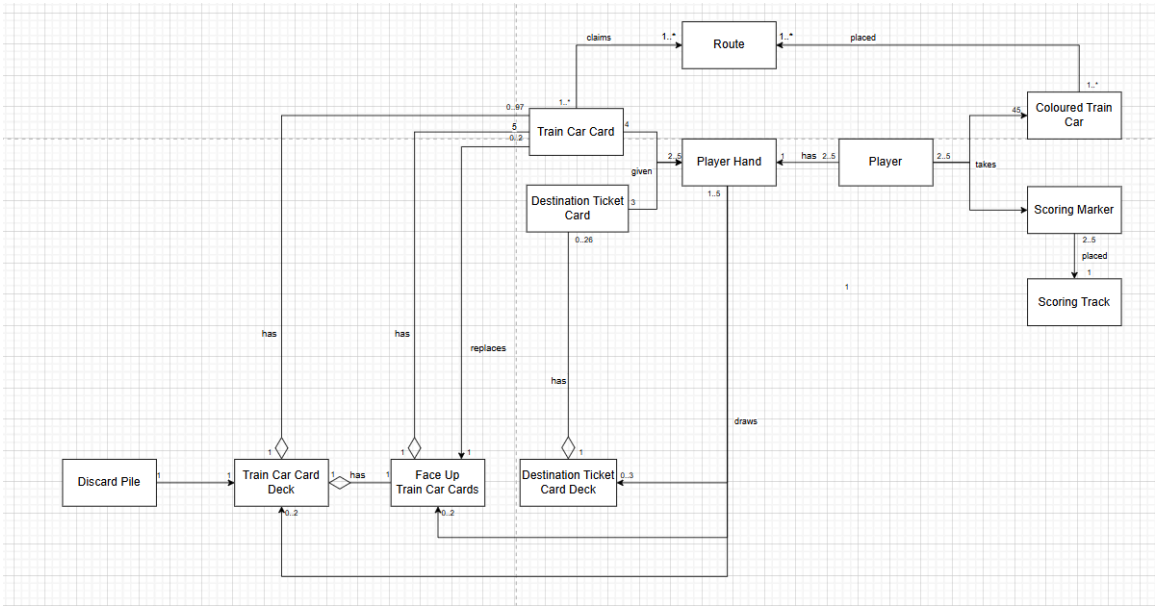

Figure 5.1.1 - The first iteration of the Domain Model

This was the first iteration of the Domain Model. It was rejected because many of the cardinalities did not make sense and also many entities were missing. For instance, there is a relationship between Player Hand and Destination Ticker Card Deck that says they may draw 0 to 3 of the Deck. This does not make sense, as the Player draws 0 to 3 Destination Ticket Cards in a turn, not decks. Some relationships are also missing. Players can hold any amount of cards, so there should be a relationship between PlayerHand and each type of card. Many key entities like City, Game Board, Game, Continuous Path etc are missing, which is bad because they are important aspects of the game and have relationships with the entities present. The types of Train Car Cards and colours of Train cars are also not represented here.

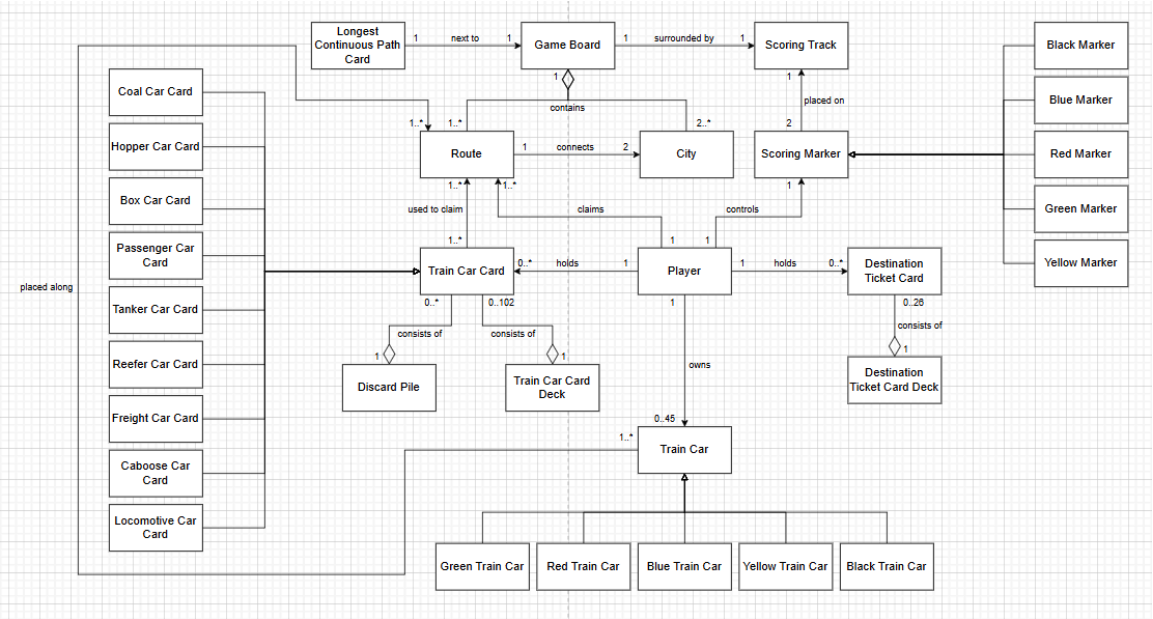

Figure 5.1.2 - The second iteration of the Domain Model

This second iteration is very close to the final product. One of the reasons it was dismissed was because it did not fully capture the entire problem space. For example, the Ticket To Ride guide explains the specific number of each type of Train Car Card, such as there being 14 Locomotive Cards. This seems like important information that should be captured in the Domain Model. In addition, we can further abstract the Train Car Card and Destination Ticket Card. Both are Cards, and so we can use a generalisation relationship from those two to a new entity named Card. Also, the Longest Continuous Card entity is not relevant in a digital version of Ticket To Ride as it simply awards bonus points to the player with the longest continuous path. Continuous Path entity is missing here, and so adding both a Continuous Path entity and a Longest Continuous Path entity, we can show how there is only one Longest Continuous Path. Likewise, the Scoring Marker and Scoring Track are entities only relevant for the physical models, and likely will not be in the digital ones, and so are unnecessary. Instead, a Point entity would make more sense. In addition, it omits all turn behaviour which was modelled in Figure 5.1.1 like drawing Train Car Cards, which are still important aspects of the gameplay needed to be represented.

Gen AI declaration

Tools used:

Chatgpt 5.2(Neil) - Prompted with a description of the ticket to ride game to generate a basic structure of a terminal based prototype. Which I built on and used as my prototype.